



معماری کامپیوتر

نیم سال ۰۴-۰۵

استاد اسدی

اعضای گروه: سید محمد حسن حسینی

طاها صمدی

علی سلطانی

سامیار صفری

دانشکده مهندسی کامپیوتر

گزارش پروژه نهایی: طراحی پردازنده MIPS با قابلیت راه اندازی امن

۱ معماری سیستم

معماری طراحی شده در این پروژه، یک پردازنده MIPS سفارشی سازی شده است که به منظور مقابله با حملات نرم افزاری و اجرای کدهای مخرب، به مکانیزم «راه اندازی امن»^۱ مجهز شده است. در این سیستم، حافظه به دو بخش کاملاً مجزا تقسیم شده است: حافظه غیر قابل تغییر Boot ROM (محل قرار گیری کدهای امنیتی و تأیید هویت) و حافظه User RAM (محل قرار گیری کدهای کاربر). پردازنده در لحظه روشن شدن، منحصرأ در حالت امن^۲ راه اندازی شده و وظیفه دارد پیش از اعطای مجوز اجرا به برنامه کاربر، صحت و یکپارچگی کدهای آن را از طریق محاسبه هش سخت افزاری تضمین نماید.

۲ نحوه پیاده سازی Mode_Bit

مفهوم Mode_Bit به عنوان هسته اصلی امنیت در این پردازنده عمل می کند. این سیگنال یک فلیپ فلاپ یک بیتی در واحد کنترل^۳ است که سطح دسترسی پردازنده را تعیین می کند. هنگامی که $Mode_Bit = 1$ است (حالت امن)، پردازنده فقط از Boot ROM دستور می خواند و دسترسی کاملی به شتاب دهنده رمز نگاری دارد. تغییر وضعیت این بیت به 0 (حالت کاربر) تنها از طریق اجرای دستور اختصاصی DROP_PRIV امکان پذیر است که یک طرفه بوده و بازگشت به حالت امن را غیر ممکن می سازد.

بخش پیاده سازی Mode_Bit در واحد کنترل:

```

1 module Controller (
2     input clk, rst,
3     input [5:0] opcode,
4     output reg Mode_Bit
5 );
6     localparam DROP_PRIV_OP = 6'b111111;
7
8     always @(posedge clk or posedge rst) begin
9         if (rst) begin
10             Mode_Bit <= 1'b1;
11         end else if (opcode == DROP_PRIV_OP) begin
12             Mode_Bit <= 1'b0;
13         end

```

^۱Secure Boot^۲Secure Mode^۳Controllor

```

14     end
15 endmodule

```

۳ طراحی Router Memory

ماژول Memory Router به عنوان دروازه بان امنیتی^۴ در مسیر داده‌ها عمل می‌کند. وظیفه اصلی این واحد، جلوگیری از دسترسی غیرمجاز برنامه کاربر (زمانی که Mode_Bit = 0 است) به آدرس‌های حساس Boot ROM و رجیسترهای شتاب‌دهنده است. این ماژول بر اساس مقدار Mode_Bit، سیگنال‌های فعال‌ساز خواندن و نوشتن^۵ را برای حافظه‌های مختلف مسیریابی و در صورت لزوم مسدود می‌کند.

توصیف سخت‌افزاری Router Memory:

```

1 module Memory_Router (
2     input Mode_Bit,
3     input MemRead_in, MemWrite_in,
4     input [31:0] address,
5     output reg ROM_Read,
6     output reg RAM_Read, RAM_Write,
7     output reg Crypto_Read, Crypto_Write
8 );
9     always @(*) begin
10         ROM_Read = 0; RAM_Read = 0; RAM_Write = 0;
11         Crypto_Read = 0; Crypto_Write = 0;
12
13         if (Mode_Bit == 1'b1) begin
14             if (address >= 32'h00000000 && address < 32'h00001000) begin
15                 ROM_Read = MemRead_in;
16             end else if (address >= 32'h00001000 && address < 32'h00002000)
17                 begin
18                     RAM_Read = MemRead_in;
19                     RAM_Write = MemWrite_in;
20                 end else if (address >= 32'h00002000) begin
21                     Crypto_Read = MemRead_in;
22                     Crypto_Write = MemWrite_in;
23                 end
24             end else begin
25                 if (address >= 32'h00001000 && address < 32'h00002000) begin
26                     RAM_Read = MemRead_in;
27                     RAM_Write = MemWrite_in;
28                 end
29             end
30         end
31     end
32 endmodule

```

^۴Security Gateway

^۵Read/Write Enables

۴ طراحی واحد هش (Crypto Accelerator)

این واحد سخت‌افزاری، یک شتاب‌دهنده ساده برای محاسبه سریع هش برنامه موجود در RAM است. پردازنده در چرخه بوت، کلمات برنامه کاربر را به ترتیب خوانده و در این واحد می‌نویسد. واحد هش با استفاده از عملیات XOR و شیفت (به عنوان یک الگوریتم نمادین و سبک در سطح سخت‌افزار)، مقدار هش نهایی را انباشت و محاسبه می‌کند تا در انتها توسط پردازنده خوانده شود.

توصیف سخت‌افزاری Crypto Accelerator:

```

1 module Crypto_Accelerator (
2     input clk, rst,
3     input Crypto_Write, Crypto_Read,
4     input [31:0] data_in,
5     output [31:0] hash_out
6 );
7     reg [31:0] internal_hash;
8
9     always @(posedge clk or posedge rst) begin
10         if (rst) begin
11             internal_hash <= 32'h12345678;
12         end else if (Crypto_Write) begin
13             internal_hash <= internal_hash ^ data_in;
14         end
15     end
16
17     assign hash_out = internal_hash;
18 endmodule

```

۵ نتایج آزمایش‌ها

۱.۵ کدهای اسمبلی مورد استفاده

پیش از بررسی سناریوها، کدهای اسمبلی نوشته شده برای راه‌انداز امن^۶ و برنامه کاربر^۷ ارائه می‌گردد.

کد اسمبلی بارگذاری شده در Boot ROM: این کد وظیفه واکشی ۱۷ دستور از حافظه RAM، تزریق آن‌ها به واحد هش، و در نهایت مقایسه هش تولیدی با هش معتبر را بر عهده دارد. در صورت برابری، دسترسی کاهش یافته و برنامه کاربر اجرا می‌شود؛ در غیر این صورت سیستم قفل^۸ می‌گردد.

```

1     addi $t0, $zero, 0x1000
2     addi $t1, $zero, 17
3     addi $t2, $zero, 0x2000
4 hash_loop:
5     lw   $t3, 0($t0)
6     sw   $t3, 0($t2)
7     addi $t0, $t0, 4
8     addi $t1, $t1, -1
9     bne  $t1, $zero, hash_loop
10

```

^۶Boot ROM

^۷User Program

^۸Halt

```

11    lw    $t4, 4($t2)
12    lw    $t5, 0x0100($zero)
13    bne   $t4, $t5, halt
14
15    drop_priv
16    j     0x1000
17
18 halt:
19    j     halt

```

کد اسمبلی نمونه برنامه کاربر: این یک برنامه ساده است که در فضای RAM (آدرس 0x1000) قرار می‌گیرد و تنها عملیات جمع و ذخیره در حافظه را انجام می‌دهد. در سناریوی دوم، یکی از مقادیر این برنامه توسط هکر دستکاری شده است.

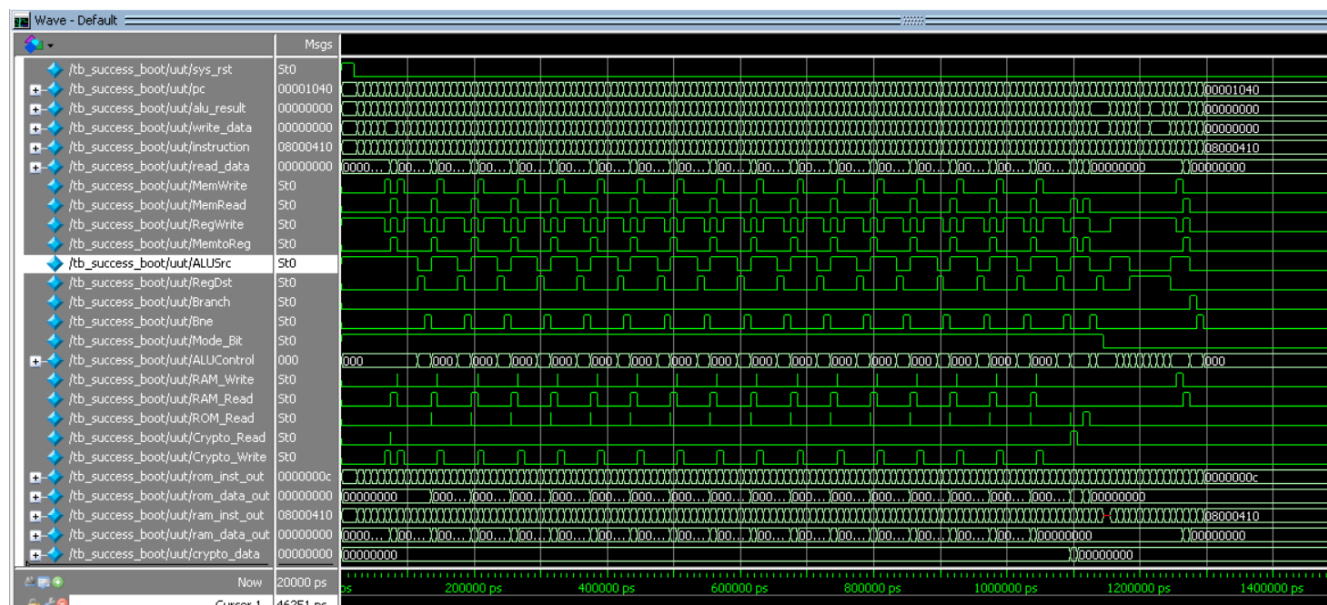
```

1    addi $s0, $zero, 5
2    addi $s1, $zero, 10
3    add  $s2, $s0, $s1
4    sw   $s2, 0x1020($zero)
5 label_end:
6    j    label_end

```

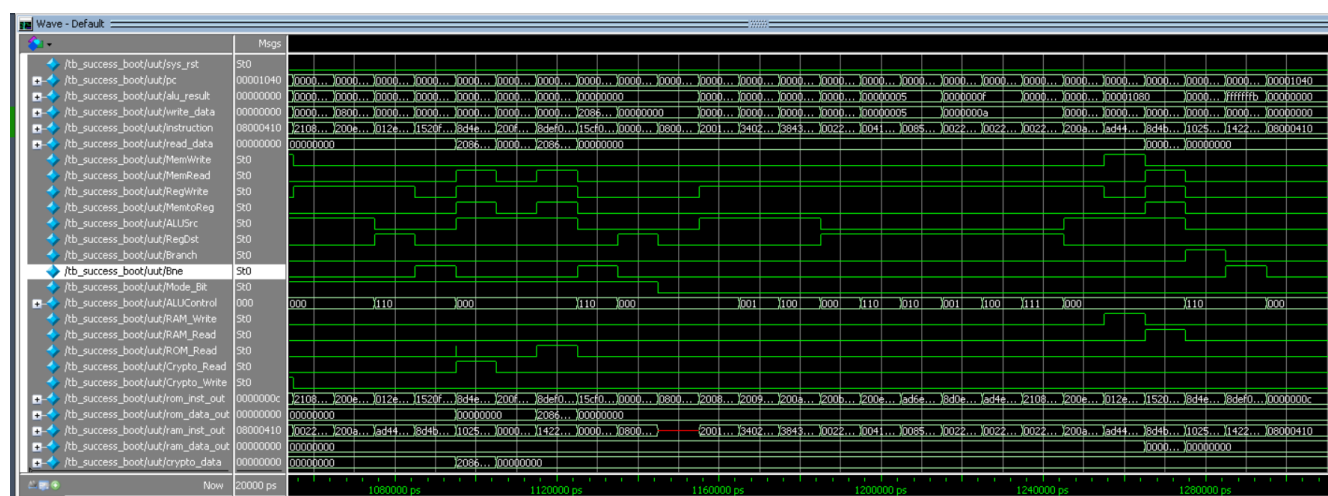
۲.۵ سناریوی اول: بوت موفق (اجرای برنامه معتبر)

در این سناریو، محتوای User RAM کاملاً دست‌نخورده و معتبر است. پردازنده پس از محاسبه هش و اطمینان از صحت کدها، با موفقیت سطح دسترسی را کاهش داده (Mode_Bit = 0) و وارد فاز اجرای کدهای کاربر می‌شود.



شکل ۱: نمای کلی از چرخه حیات بوت موفق

تحلیل نمای کلی موج: این تصویر یک نمای حماسی و کامل از کل روند پردازش را نشان می‌دهد. پردازنده از ابتدا تا انتها به درستی کار کرده، حلقه هش تکرار شده و در نهایت سیگنال Mode_Bit در انتهای کار افت کرده و سیستم با موفقیت وارد فاز اجرای برنامه کاربر در RAM شده است.



شکل ۲: نمای دقیقی از فاز انتقال دسترسی و پرش به برنامه کاربر

تحلیل نمای زوم شده: این بخش، حساس ترین مقطع سناریوی اول است. در حوالی زمان 1,100,000 ps پالس Crypto_Read برای خواندن هش نهایی فعال می شود. پس از تایید برابری هش ها در ALU، سیگنال Mode_Bit در 1,135,000 ps با موفقیت به صفر تغییر وضعیت می دهد.^۹ بلافاصله مقدار شمارنده برنامه (PC) با یک پرش دقیق به آدرس 00001000 (نقطه شروع برنامه کاربر) منتقل شده و در نهایت برنامه کاربر با فعال شدن RAM_Write با موفقیت خاتمه می یابد.

لاگ خروجی موفقیت آمیز ModelSim در سناریوی اول:

```

1 # Time = 20000 | PC = 00000000 | Mode_Bit = 1 | Watchdog_Rst = 0
2 # Time = 25000 | PC = 00000004 | Mode_Bit = 1 | Watchdog_Rst = 0
3 # ...
4 # Time = 1085000 | PC = 0000002c | Mode_Bit = 1 | Watchdog_Rst = 0
5 # Time = 1095000 | PC = 00000030 | Mode_Bit = 1 | Watchdog_Rst = 0
6 # Time = 1105000 | PC = 00000034 | Mode_Bit = 1 | Watchdog_Rst = 0
7 # Time = 1115000 | PC = 00000038 | Mode_Bit = 1 | Watchdog_Rst = 0
8 # Time = 1125000 | PC = 0000003c | Mode_Bit = 1 | Watchdog_Rst = 0
9 # Time = 1135000 | PC = 00000040 | Mode_Bit = 0 | Watchdog_Rst = 0
10 # Time = 1145000 | PC = 00001000 | Mode_Bit = 0 | Watchdog_Rst = 0
11 # Time = 1155000 | PC = 00001004 | Mode_Bit = 0 | Watchdog_Rst = 0
12 # -----
13 # --- Test Completed: Secure Boot Successful ---
14 # -----

```

تست پنچ سناریوی بوت موفق:

```

1 module tb_success_boot;
2     reg clk, rst_button;
3
4     Top_System uut (
5         .clk(clk),
6         .rst_button(rst_button)
7     );
8

```

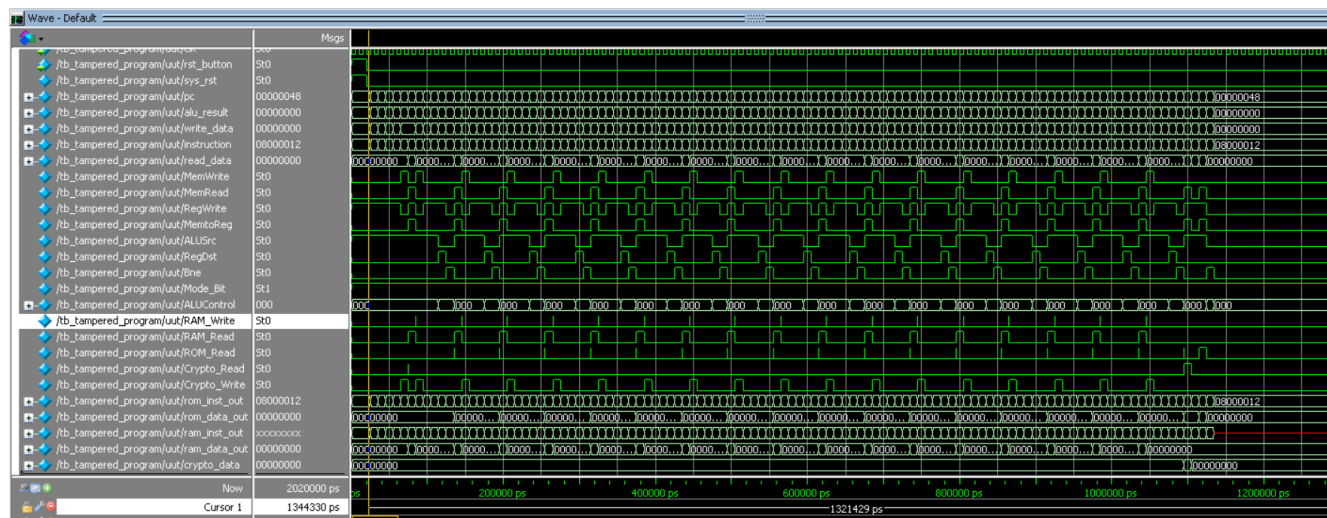
```

9      always #5000 clk = ~clk;
10
11      initial begin
12          clk = 1; rst_button = 1;
13          #10000 rst_button = 0;
14
15          $readmemh("boot_rom.hex", uut.boot_rom_inst.memory);
16          $readmemh("user_program_valid.hex", uut.user_ram_inst.memory);
17
18          #1400000 $stop;
19      end
20 endmodule

```

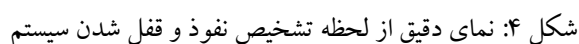
۳.۵ سناریوی دوم: بوت ناموفق (برنامه دستکاری شده توسط هکر)

در این سناریو، فرض بر این است که یک بدافزار یا هکر توانسته یک عدد را در User RAM دستکاری کند (تغییر عدد ۵ به ۶ در کد ماشین). سیستم باید این تغییر را تشخیص داده و از اجرای کد و خروج از حالت امن جلوگیری کند.



شکل ۳: نمای کلی از موج در سناریوی برنامه دستکاری شده

تحلیل نمای کلی موج: بارزترین نکته در این تصویر، خط ممتد و بالای سیگنال `Mode_Bit` از ابتدا تا انتهای شبیه سازی است. برخلاف سناریوی اول، سیستم هرگز فریب نمی خورد. پردازنده پس از کشف دستکاری، ارتباط با User RAM را کاملاً مسدود کرده و در سمت راست تصویر، هیچ گونه پالس فعالی روی `RAM_Read` یا `RAM_Write` مشاهده نمی شود.



تحلیل نمای زوم‌شده: این تصویر سند اثبات امنیت سخت‌افزاری پردازنده است. پس از خواندن هشت نهای (Crypto_Read) در زمان 1,090,000 ps، واحد ALU متوجه عدم برابری هشت‌ها می‌شود و سیگنال پرش شرطی (Bne) فعال می‌گردد. در زمان 1,135,000 ps، به جای اجرای دستور DROP_PRIV، شمارنده برنامه (PC) مستقیماً به آدرس 00000048 پرش می‌کند که آدرس حلقه بی‌نهایت^{۱۱} است. سیگنال Mode Bit محکم روی 1 باقی مانده و سیستم برای همیشه در حالت امن قرنطینه می‌شود.

لاگ خروجی ModelSim در سناریوی برنامه دستکاری شده:

```

1 # Time = 20000 | PC = 00000000 | Mode_Bit = 1 | Watchdog_Rst = 0
2 # Time = 25000 | PC = 00000004 | Mode_Bit = 1 | Watchdog_Rst = 0
3 # ...
4 # Time = 1085000 | PC = 0000002c | Mode_Bit = 1 | Watchdog_Rst = 0
5 # Time = 1095000 | PC = 00000030 | Mode_Bit = 1 | Watchdog_Rst = 0
6 # Time = 1105000 | PC = 00000034 | Mode_Bit = 1 | Watchdog_Rst = 0
7 # Time = 1115000 | PC = 00000038 | Mode_Bit = 1 | Watchdog_Rst = 0
8 # Time = 1125000 | PC = 0000003c | Mode_Bit = 1 | Watchdog_Rst = 0
9 # Time = 1135000 | PC = 00000048 | Mode_Bit = 1 | Watchdog_Rst = 0
10 # -----
11 # --- Test Completed: Tampered Program Scenario ---
12 # -----
13 # Break in Module tb tampered_program at tb_tampered_program.v line 32

```

تست پنج سناریوی برنامه دستکاری شده (هکر):

```
1 module tb_tampered_program;
2     reg clk, rst_button;
3
4     Top_System uut (
5         .clk(clk),
6         .rst_button(rst_button)
7     );
8
```

```

9      always #5000 clk = ~clk;
10
11      initial begin
12          clk = 1; rst_button = 1;
13          #10000 rst_button = 0;
14
15          $readmemh("boot_rom.hex", uut.boot_rom_inst.memory);
16          $readmemh("user_program_tampered.hex", uut.user_ram_inst.memory);
17
18          #1400000 $stop;
19      end
20 endmodule

```

۶ چالش‌ها و مشکلات پروژه

طراحی یک پردازنده امنیتی در سطح RTL با چالش‌های متعددی همراه بود که با تحلیل دقیق شکل موج‌ها برطرف گردیدند:

- **چالش حلقه ترکیبی^{۱۱} و خطای X:** در مراحل اولیه، استفاده از حافظه‌های تک‌پورته باعث تداخل مسیر واکنشی دستور و خواندن داده (lw) می‌شد که به ایجاد حلقه ترکیبی و نامعلوم شدن وضعیت PC می‌انجامید. برای حل این مشکل، معماری ماژول‌های حافظه (Boot_ROM و User_RAM) به طور کامل بازنویسی شده و به حافظه‌های دو پورته^{۱۲} ارتقا یافتند تا تداخل مسیرها به طور کامل برطرف شود.
- **بن‌بست واکنشی دستور پس از کاهش سطح دسترسی:** در سناریوی بوت موفق، پس از اجرای دستور اختصاصی DROP_PRIV، مالتی‌پلکسر واکنشی دستور به دلیل وابستگی سخت‌گیرانه به Mode_Bit، مانع از خوانده شدن دستور پرش به RAM می‌شد و پردازنده در وضعیت اجرای دستورات NOP سر می‌خورد^{۱۳}. با اصلاح مسیر واکنشی دستور در Top_System و واگذاری مسئولیت امنیتی به Memory_Router، این بن‌بست برطرف شد.
- **مدیریت نسخه و سازگاری فایل‌های شبیه‌ساز^{۱۴}:** در زمان همگام‌سازی کدهای پروژه در GitHub، فایل‌های موقتی و باینری ModelSim (مانند پوشه work) باعث تداخل بین سیستم‌عامل‌های مختلف می‌شد. این مشکل با پیاده‌سازی یک فایل .gitignore استاندارد و حذف فایل‌های Cache شده برطرف گردید.

^{۱۱}Combinational Loop

^{۱۲}Dual-Port Memory

^{۱۳}NOP Slide

^{۱۴}Version Control